

Machine Learning

Course Project Phase 1&2

Title of the project: Electrical Grid Stability

Student: Name - Rishi Raj N

Email - rishiraj.n-26@scds.saiuniversity.edu.in

ML Category: Classification

1) Introduction

The examination of local stability of a 4-node star network in a Decentral Smart Grid Control architecture is a technique to ensure that, despite supply or demand changes, the power distribution from a central power producer to four connected nodes (consumers) remains stable. This system utilises decentralised smart grid control, a smart technology, to facilitate communication between each node and the master node, helping to maintain equilibrium and prevent power grid interruptions.

2) Dataset and Features

The analysis is performed for various sets of input (12 Features) as follows: tau1, tau2, tau3, tau4, p1, p2, p3, p4, g1, g2, g3 and g4, to find the target variables stab and stabf.

- ☐ tau[x]: the participant's reaction time (real number within the range of [0.5, 10] seconds). tau1 - the value for a source's electricity.
- ☐ p[x]: $\frac{\text{Estimated power consumed (negative)}}{\text{produced (positive) (real number)}}$, for consumers, the range is $[-0.5, -2] s^{-2}$; where $p1 = |(p2 + p3 + p4)|$.
- ☐ g[x]: the coefficient (gamma) relating to the real price elasticity (from the interval $[0.05, 1] s^{-1}$). g1 is the value for a source's power.
- ☐ stab: The maximum real part of the characteristic equation root. If positive, the system is linearly unstable (real number).
- ☐ stabf: The stability label of the system (categorical: stable/unstable).

All of the variables are continuous except for the stabf which is binary. There are 10,000 data instances available for different input combinations. In this project, we will utilise the 12 features (X) and focus only on the stabf (y) for our dataset to model it using different ML techniques. Moreover, scaled data and the default Scikit-Learn library parameters will be used for all modelling. The models' random state parameter isn't specified because they are being run with their default settings, which will cause the model to output a variety of values with just minor differences.

3) Methods

3.1) Baseline - Logistic Regression:

Our dataset presents a binary classification scenario where we aim to identify whether the system is stable or unstable. To address this, the Logistic Regression algorithm is a fitting choice. Logistic regression is a commonly employed method for classifying data by

predicting the probability of an instance belonging to a specific class. If the estimated probability exceeds 50%, the model predicts the instance belongs to class 1; if it falls below 50%, it predicts the instance belongs to class 0.

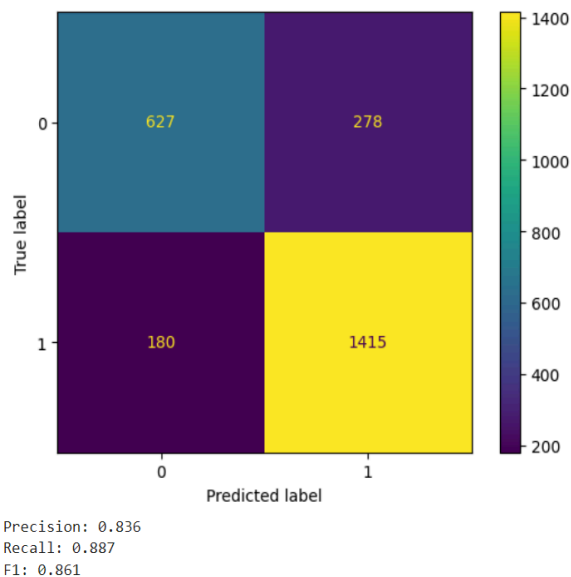
The performance of the classification model on the test data set is shown in the confusion matrix. The confusion matrix gives a detailed understanding of a model's recall, accuracy, precision, and overall effectiveness in distinguishing between classes by examining true positive, true negative, false positive, and false negative predictions.

$$\begin{aligned} 1) \text{ Precision} &= \frac{TP}{TP+FP} \\ 2) \text{ Recall} &= \frac{TP}{TP+FN} \\ 3) \text{ F1 score} &= 2 \times \frac{\text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} \end{aligned}$$

We will use the F1 score values to compare accuracy with other models and create a bar graph to visually represent it.

Results:

The Confusion Matrix for Logistic Regression:



The F1 score for logistic regression is 0.861.

3.2) Support Vector Machines:

The Support Vector Machine (SVM) stands out as a robust and flexible machine learning model with the ability to handle both linear and nonlinear regression as well as classification tasks. Its main goal is to determine the optimal hyperplane within an N-dimensional space to effectively distinguish data points of different classes within the feature space. A Support

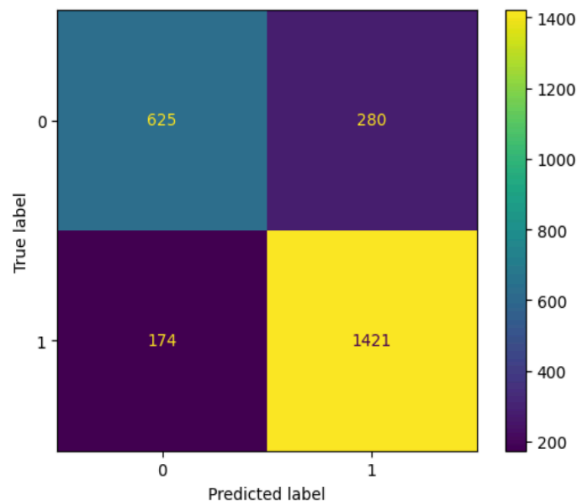
Vector Machine (SVM) kernel is a function that transforms low-dimensional input space into a higher-dimensional space. It includes various types of kernel functions such as radial basis function (RBF), polynomial, and linear.

Results:

The F1 score for SVM Linear is 0.86, SVM RBF is 0.97 and SVM Poly is 0.91.

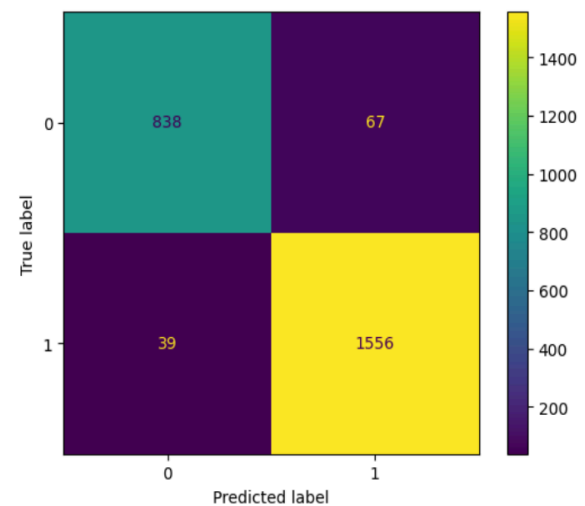
The confusion matrix for all the SVM's are as follows:

SVM - Linear Kernel:



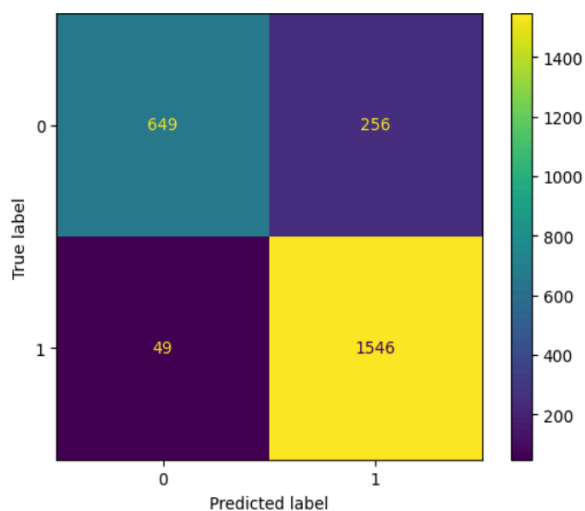
Precision: 0.835
Recall: 0.891
F1_Lin: 0.862

SVM - RBF Kernel:



Precision: 0.959
Recall: 0.976
F1_rbf: 0.967

SVM - Polynomial Kernel:



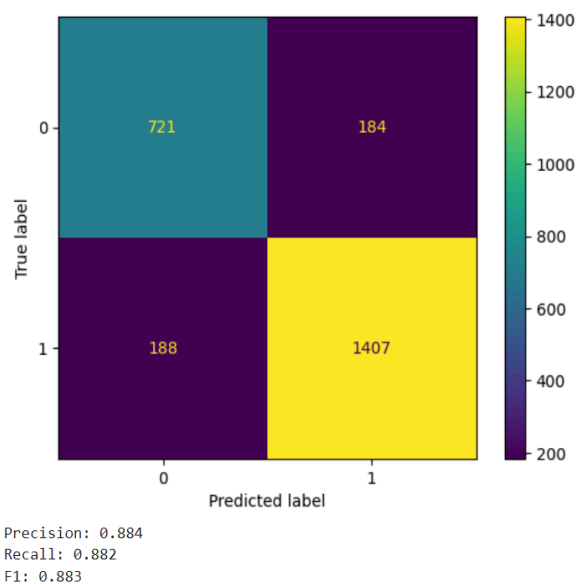
Precision: 0.858
Recall: 0.969
F1_poly: 0.910

3.3) Decision Tree:

The decision tree is a non-parametric supervised learning approach used for classification and regression tasks. Its structure resembles a tree, with each internal node testing an attribute, each branch denoting an attribute value, and each leaf node representing the conclusion or forecast. The decision tree is trained using the CART algorithm, and feature scaling is not necessary for modelling.

Results:

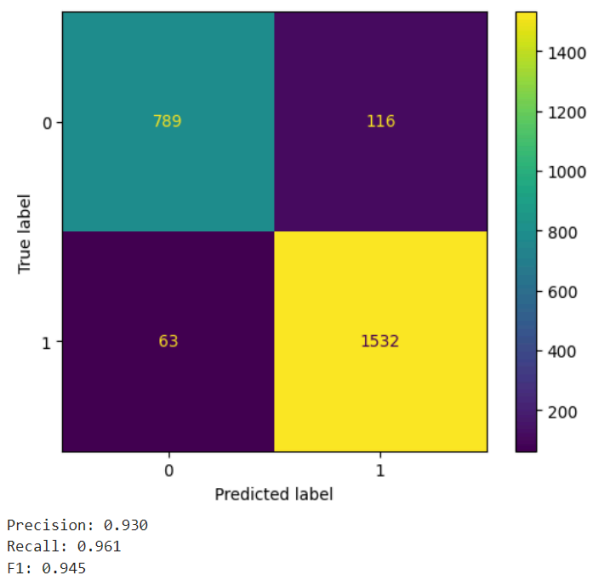
The F1 score for Decision tree is 0.883.



3.4) Random Forest:

Random Forest is an ensemble learning technique used for classification and regression. During training, it builds a collection of decision trees and produces a class that is the mean prediction for regression or the mode of the classes for classification from each individual tree. It is considered one of the best models. Training can be slow, particularly when dealing with big datasets or a lot of trees.

Result: The F1 score for Random Forest is 0.945.

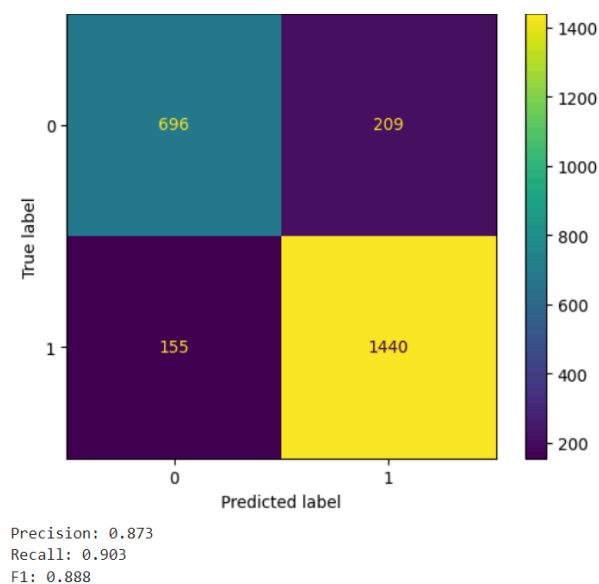


3.5) AdaBoost:

AdaBoost, an ensemble learning method, constructs a powerful classifier by amalgamating multiple weak classifiers. Training is performed sequentially, utilising the performance of previously trained learners to adjust weaker learners subsequently. They are sensitive to noisy data and outliers.

Result:

The F1 score for AdaBoost is 0.888.

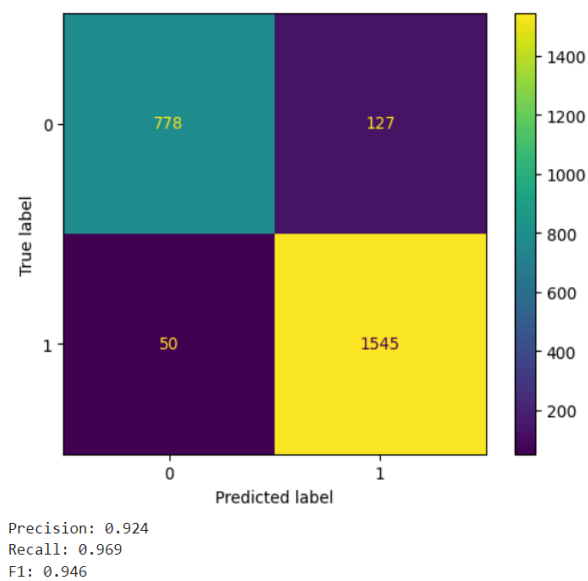


3.6) Gradient Boosting:

The gradient boosting method is an ensemble learning technique that builds models sequentially. It corrects the residual errors of the previous predictor by creating new ones. Although it is computationally demanding and requires careful calibration, it offers great accuracy.

Result:

The F1 score for Gradient Boosting is 0.946.

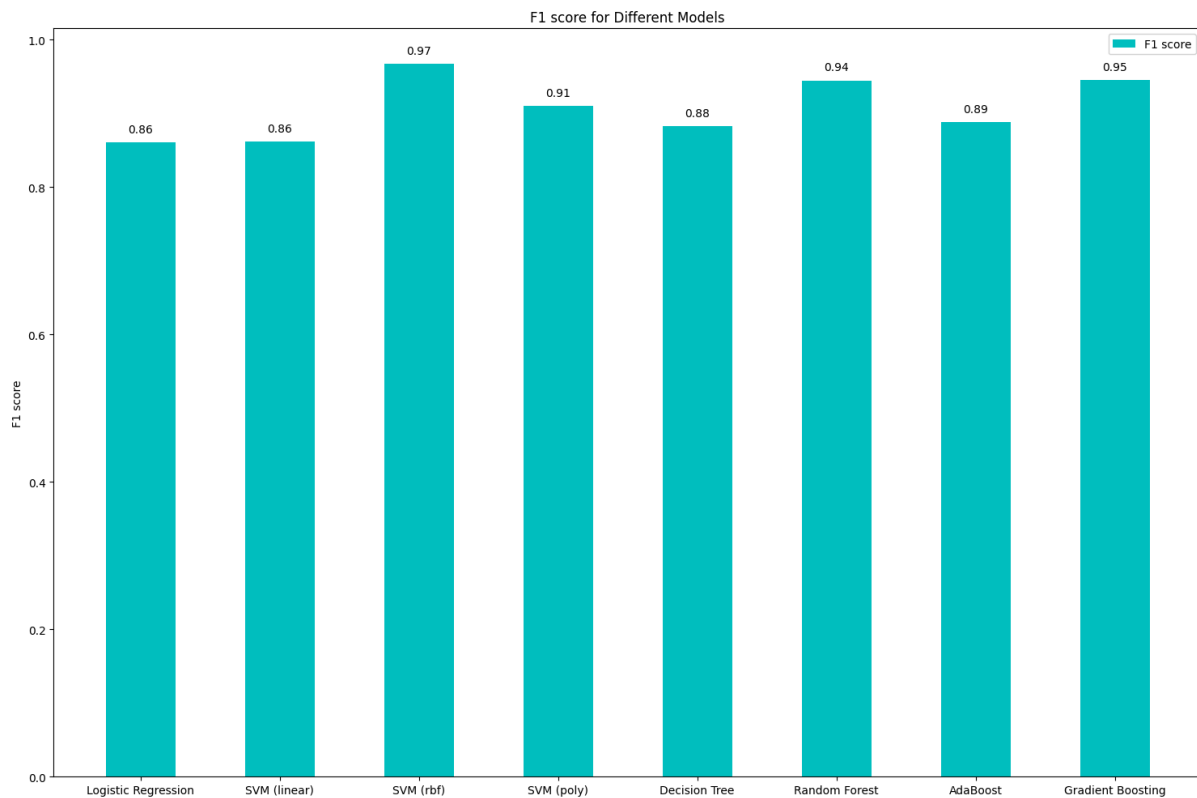


4) Results

Here is the tabular representation of the F1 scores for all the different models:

Model		F1 scores
Logistic Regression (Baseline)		0.86
Support Vector Machine	Liner	0.86
	RBF	0.97
	Polynomial	0.91
Decision Tree		0.88
Random Forest		0.94
AdaBoost		0.89
Gradient Boosting		0.95

The bar graph visualisation below displays all the different models with their corresponding F1 score.



Based on the results, the SVM (RBF) produced the most accurate results, achieving 97% accuracy in classifying the Electrical Grid Stability dataset. Following that, Gradient Boosting achieved 95% accuracy. In comparison to the baseline result, all models showed improved accuracy. Optimising the hyperparameters could lead to even better results.

5) Hyperparameter Tuning

The models were created using either grid search or random search, depending on the model, with 5-fold cross-validation. The best score represents the average cross-validated score for the best estimator, and the F1 score of the models was computed along with the optimal values for the parameters.

5.1) SVM with RBF Kernel:

The following hyperparameters were used to tune in this model: C and gamma.

- C behaviours in an SVM regularisation parameter. Higher C values may allow for a narrower margin if the decision function performs better at correctly classifying all training points. Despite the potential compromise in training accuracy, a lower C value will encourage a larger margin and a simpler decision function.

- The gamma parameter indicates how far a single training sample can influence something. Low values indicate "far," while high values indicate "close."

Grid search was conducted using a parameter grid for this model. The ranges for the parameters are,

- ☐ 'C':[i for i in range(1,10)].
- ☐ 'gamma': np.linspace(0.1, 1, 10).

Results:

- The Best parameter values are C - 9 and gamma - 0.1.
- The best score is 0.9596.
- The F1 score after modelling with optimal values is 0.973.

5.2) Decision Tree:

The following hyperparameters were tuned in this model:

- max_depth: The maximum depth of the tree. If None, then nodes are expanded until all leaves are pure or until all leaves contain less than min_samples_split samples.
- max_features: The number of features to consider when looking for the best split. If int, then consider max_features features at each split.
- max_leaf_nodes: The maximum number of leaf nodes present. If None then unlimited number of leaf nodes.
- min_samples_leaf: The minimal number of samples necessary for a leaf node to exist.
- min_samples_split: The lowest number of samples needed for splitting an internal node.

Random search was conducted using a parameter distribution for this model because there are more than 2 parameters to be tuned. The range for the parameters are as follows:

- ☐ 'max_depth':[i for i in range(5,51,5)].
- ☐ 'max_features':[i for i in range(1,13)].
- ☐ 'max_leaf_nodes':[i for i in range(5,51,5)].
- ☐ 'min_samples_leaf':[i for i in range(2,21,2)].
- ☐ 'min_samples_split':[i for i in range(2,21,2)].

Results:

- The best parameters values are max_depth = 35, max_features = 11, max_left_nodes = 50, min_samples_leaf = 8 and min_samples_split = 10.
- The best score is 0.840.
- The F1 score after tuning is 0.885.

5.3) Random Forest:

The model's hyperparameters were tuned similarly to the decision tree model. These hyperparameters include `max_depth`, `max_leaf_nodes`, `max_features`, `min_samples_leaf`, and `n_estimators`. The `n_estimators` hyperparameter determines the number of trees in the forest and has a default value of 100.

A random search was conducted using parameter distribution for this model. The range for the parameters were the same as for the decision tree and for `n_estimators`,

□ `'n_estimators': [i for i in range(100,151,10)]`.

Results:

- The best parameter values are `n_estimators = 130`, `max_depth = 30`, `max_features = 5`, `max_leaf_nodes = 50`, `min_samples_leaf = 6` and `min_samples_split = 8`.
- The Best score is 0.890.
- The F1 score after tuning is 0.921.

5.4) Ada Boost:

The hyperparameters, `n_estimators` and `learning_rate`, were adjusted to optimise the AdaBoost classifier model.

- `n_estimators` - This refers to the maximum number of estimators used before the boosting process is stopped.
- `learning_rate` - This denotes the weight applied to each classifier at every boosting iteration. A higher learning rate increases the contribution of each classifier.

The grid search was performed using a parameter grid for this model. The ranges for the parameters are:

□ `'n_estimators': [i for i in range(100,151,10)]`.

□ `'learning_rate': np.linspace(0.1, 1, 10)`.

Results:

- The best parameter values are `n_estimators = 140` and `learning_rate = 0.5`.
- The best score is 0.850.
- The F1 score after tuning is 0.895.

5.5) Gradient Boosting:

The hyperparameters, n_estimators, learning rate and max_depth were adjusted to optimise the Gradient Boosting classifier model. A random search was conducted using parameter distribution for Gradient Boosting classifier.

The range for the parameters were set to:

- ☐ 'n_estimators':[i for i in range(100,151,10)],
- ☐ 'learning_rate':np.linspace(0.1, 1, 10),
- ☐ 'max_depth':[i for i in range(10,101,10)]

Results:

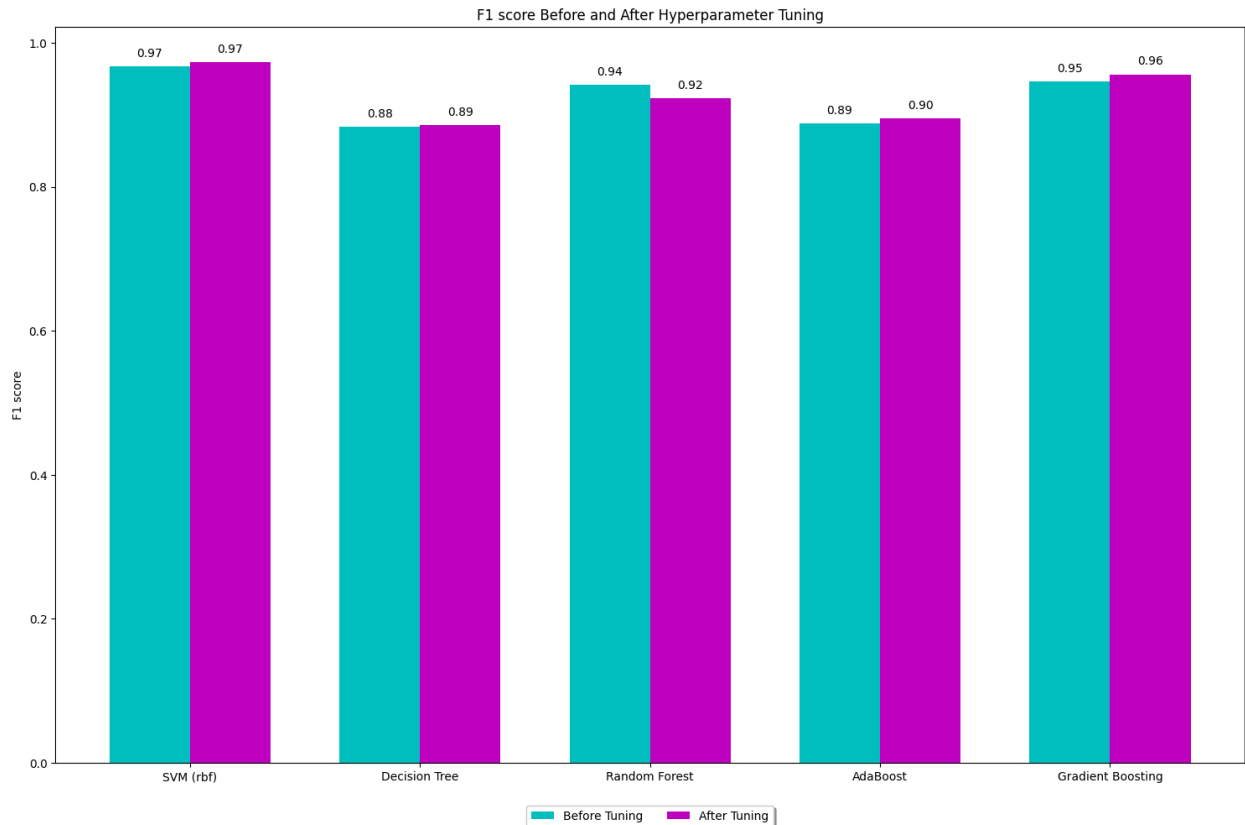
- The best parameters value are n_estimators = 130, learning rate = 0.5 and max_depth = 10.
- The best score is 0.922.
- The F1 score after tuning is 0.956.

6) Results after hyperparameter tuning

6.1) Tabular Representation of the results:

Model	F1 scores before tuning	F1 scores after tuning
SVM with RBF Kernel	0.97	0.97
Decision Tree	0.88	0.89
Random Forest	0.94	0.92
AdaBoost	0.89	0.90
Gradient Boosting	0.95	0.96

6.2) Bar Visualization of F1 scores of models before and after tuning:



Based on the results, the SVM (RBF) produced the most accurate results, achieving 97% accuracy in classifying the Electrical Grid Stability dataset after hyperparameter tuning. Following that, Gradient Boosting achieved 96% accuracy. All the models exhibited minimal changes in the f1 score before and after the tuning.

7) Feature Reduction

I used the Principal Component Analysis (PCA) technique to reduce the number of features from 12 to 6, which is 50% of the original dimension. Then, I used this dataset to train the best performing model, which happened to be the Support Vector Machine (SVM) with the Radial Basis Function (RBF) kernel. Finally, I calculated the F1 score for this model.

Result: The F1 score for this model is 0.863.

8) Feature Selection

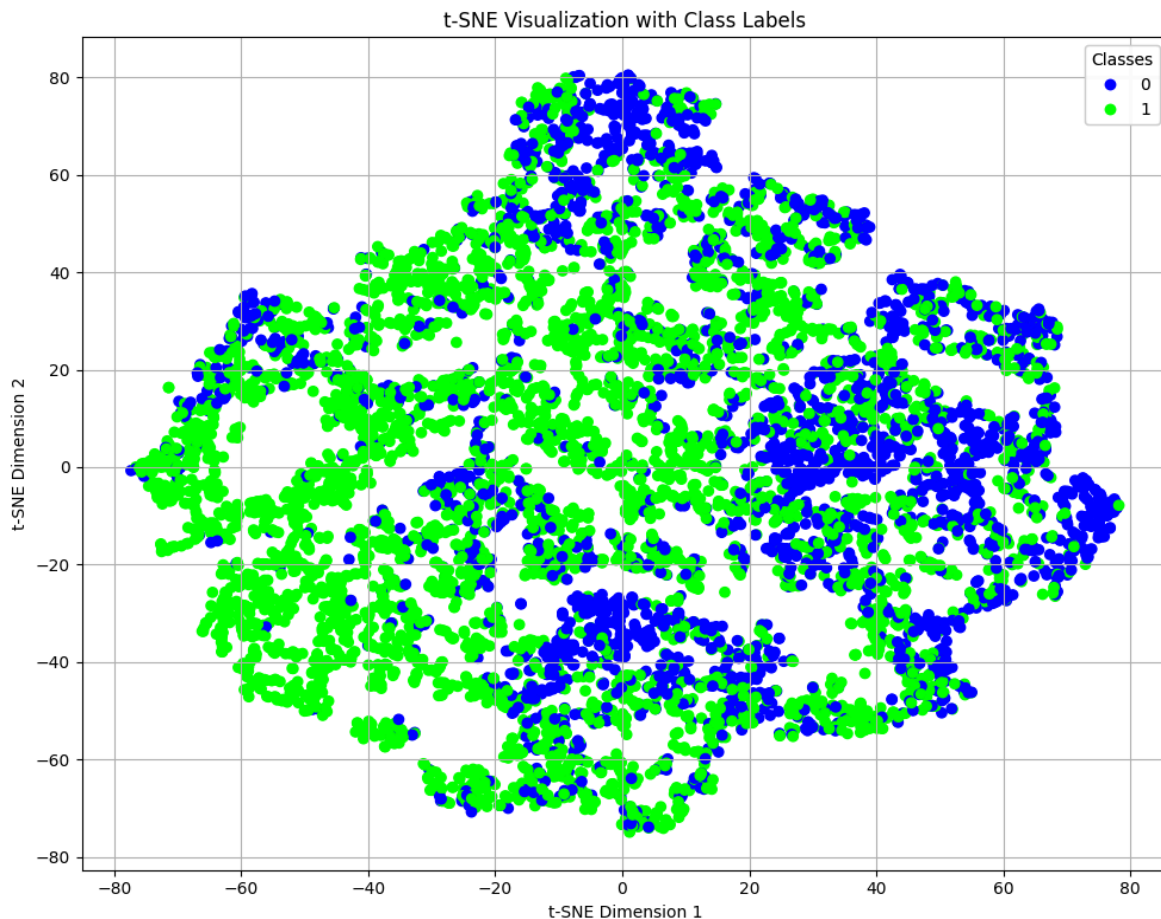
I utilised the SelectPercentile method to retain the top 50% of the highest-scoring features. These selected features were used to train an SVM model with the RBF kernel, and the F1 score was computed.

Results:

- The selected features are tau1, tau2, tau3, tau4, g2 and g3.
- The F1 score is 0.887.

9) Data Visualization

The T-SNE dimensionality reduction technique was used to reduce the features to 2D, and the visualization is as follows:



10) Conclusion

This project aimed to identify the most suitable machine learning model to classify an electrical grid system's stability, focusing on achieving high F1 score accuracy. The analysis began with the baseline model (Logistic Regression) and then proceeded to model with various classifiers including SVM with different kernels, Decision tree, Random forest, AdaBoost and Gradient Boosting.

After initial model evaluation, hyperparameter tuning was conducted to find the optimal parameter values to enhance model accuracy. There were slight improvements in the F1 score of the models after tuning. The RBF kernel achieved the highest F1 score of 0.973 with optimal parameters ($C=9$ and $\gamma=0.1$).

Compared to the baseline result, all models demonstrated improved accuracy since logistic regression was limited to linear assumptions. The RBF kernel model excelled due to its ability to handle non-linear relationships and map input into higher dimensional spaces, effectively capturing the complex patterns in the dataset. Gradient Boosting and Random Forest also performed well, demonstrating the ability of ensemble approaches to increase model accuracy and reliability.